

ArduinoEspañol — Analizador Léxico

Proyecto de Programación de Sistemas de Base 1

Campo	Valor
Materia	Programación de Sistemas de Base 1
Institución	Universidad Autónoma de Tamaulipas
Semestre	2026-1
Profesor(es)	Dr. Dante Adolfo Muñoz Quintero
Integrantes	Gómez Rubio Iram Said Reyes Avalos Joshua Emmanuel Contreras Morales Ricardo Axel Serrano Vargas Aldo Antonio
Repositorio	https://github.com/Aldo-Idk04/Proyecto-Integrador-Arduino-Espa-ol.git
Fecha	Mayo 2026

El presente documento constituye la Demostración Funcional del proyecto de semestre para la materia de Programación de Sistemas de Base 1. Incluye la especificación del lenguaje diseñado, el código fuente comentado del analizador léxico y capturas de ejecución con casos válidos y con errores.

Tabla de contenido

Contents

Proyecto de Programación de Sistemas de Base 1	1
Tabla de contenido	2
Sección 2 — Objetivos	2
Objetivo general	2
Objetivos específicos	2
Sección 3 — Especificación del Lenguaje.....	3
3.1 Nombre y propósito	3
3.2 Ejemplos ilustrativos de código	3
Configuración inicial.....	3
Bucle condicional interactivo	3
Ejemplo de control de servomotores y bucles fijos	3
3.3 Catálogo de tokens.....	4
Sección 4 — Implementación.....	6
Sección 5 — Demostración Funcional.....	10
Sección 6 — Conclusiones.....	19
Trabajo futuro (esta sección totalmente opcional)	19

Sección 1 — Introducción

El análisis léxico es la primera fase de un compilador o intérprete: recibe una secuencia de caracteres y produce una secuencia de *tokens* que representan las unidades mínimas con significado del lenguaje. Comprender esta etapa de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales que desee construir herramientas de procesamiento de lenguajes.

El presente proyecto diseña e implementa un analizador léxico para **Arduino**, un lenguaje personalizado de propósito educativo orientado a la escritura de programas de cálculo aritmético con soporte para variables, estructuras de control básicas y funciones. El nombre refleja su propósito principal: un lenguaje de scripting para cálculos.

El lexer fue implementado en Python 3.12 sin dependencias externas, usando expresiones regulares de la biblioteca estándar (`re`). Se diseñó para transformar archivos de texto fuente `.ino` en una lista numerada de tokens, e informar con precisión la línea y columna de cualquier error léxico detectado.

Este documento se organiza de la siguiente manera: la Sección 2 presenta los objetivos del proyecto; la Sección 3 describe la especificación del lenguaje Arduino; la Sección 4 contiene el código fuente comentado del lexer; la Sección 5 muestra capturas de ejecución; finalmente las Secciones 6 y 7 presentan conclusiones y referencias.

Sección 2 — Objetivos

Objetivo general

Diseñar un lenguaje personalizado denominado Arduino e implementar un analizador léxico funcional en Python que transforme programas fuente escritos en dicho lenguaje en listas de tokens, con manejo preciso de errores léxicos.

Objetivos específicos

1. Definir la especificación informal del lenguaje Arduino mediante ejemplos ilustrativos de código que cubran sus construcciones principales.
2. Establecer un catálogo completo de tokens: palabras reservadas, identificadores, números enteros y flotantes, operadores aritméticos y relacionales, delimitadores y cadenas de texto.
3. Implementar el analizador léxico usando expresiones regulares, asegurando el reconocimiento correcto de todos los tokens definidos.
4. Implementar la detección y reporte de errores léxicos, indicando número de línea, columna y descripción del problema para cada error encontrado.
5. Probar el lexer con al menos tres programas válidos y dos programas con errores léxicos que cubran los casos descritos en la especificación.

Sección 3 — Especificación del Lenguaje

3.1 Nombre y propósito

El lenguaje se denomina **ArduinoEspañol**. El nombre refleja de forma directa su propósito tecnológico y su diferenciador principal: la unificación del ecosistema de hardware abierto Arduino con la accesibilidad de la lengua española. Esto democratiza la enseñanza de la robótica y la domótica, permitiendo al estudiante enfocarse exclusivamente en la lógica de control secuencial.

3.2 Ejemplos ilustrativos de código

Configuración inicial

```
configurar pin 13
como salida

configurar pin A0
como entrada
```

Bucle condicional interactivo

```
Si leer sensor A0
> 500 entonces {
  entonces led 13
  esperar 500 ms
}
sino {
  apagar led 13
}
```

Ejemplo de control de servomotores y bucles fijos

```
repetir 5 veces {
  mover servo D9 a
90 grados
  esperar 1
segundos
  mover servo D9 a 0
grados
}
```

3.3 Catálogo de tokens

Categoría	Descripción	Patrón / Expresión regular sugerida
ACCION	Palabras clave que definen operaciones físicas sobre los componentes.	configurar encender apagar leer escribir esperar mo
COMPONENTE	Nombres de los dispositivos electrónicos soportados.	led sensor servo buzzer motor boton pantalla
PALABRA_CLAVE	Conectores gramaticales de la sintaxis estructural.	pin como en a durante veces
MODO	Modos operacionales para la configuración de pines E/S.	entrada salida entrada_pullup
VALOR_DIGITAL	Estados lógicos booleanos aplicables a señales discretas.	alto bajo
CONTROL	Estructuras de control condicional e iterativo.	si entonces sino fin_si fin_repetir
OPERADOR	Símbolos relacionales para comparaciones lógicas.	>= \ <= \ == \ != \ > \ <

Categoría	Descripción	Patrón / Expresión regular sugerida
UNIDAD	Unidades de medida para temporización y posicionamiento.	ms segundos grados
PIN_ANALOGICO	Identificador restringido de pines de entrada analógica.	A[0-5]
PIN_DIGITAL	Identificador restringido de pines digitales de la placa.	D([0-9] 1[0-3])
NUMERO	Expresiones numéricas enteras o con punto decimal.	[0-9]+(\.[0-9]+)?
CADENA	Literales de texto delimitados por comillas simples o dobles.	"[^"\\r\\n]*"
IDENTIFICADOR	Nombres de variables definidas por el usuario.	[a-zA-Z_][a-zA-Z0-9_]*
COMENTARIO	Fragmentos descriptivos descartados por el lexer.	#.*\\ /.*/\\ *\\[s\\S]*?*\\

Identificadores válidos: comienzan con letra (a-z, A-Z) o guión bajo (_), seguidos de cero o más letras, dígitos o guiones bajos.

Identificadores inválidos (generan error): comienzan con dígito (123abc), contienen caracteres especiales no permitidos (my-var, x@y).

Sección 4 — Implementación

A continuación se presenta el código fuente de los módulos del analizador léxico de ArduinoEspañol. Los fragmentos seleccionados ilustran la arquitectura y la lógica de escaneo implementada de forma manual en Python. El código completo se encuentra disponible en el repositorio de GitHub indicado en la portada.

4.1 Definición de tokens y palabras reservadas

El archivo tokens.py define la enumeración de tipos de token y la estructura de datos que representa cada unidad léxica reconocida. Utilizar una clase de constantes centralizadas garantiza que el lexer y las futuras fases del compilador compartan exactamente los mismos identificadores para las acciones, componentes, modos de pin y operadores del lenguaje.

Fragmento de src/tokens.py

```
from enum import Enum, auto
from dataclasses import dataclass
from typing import Any

class TokenTypes(Enum):
    # ACCIONES
    Configurar = auto() # configurar
    Encender = auto() # encender

    # COMPONENTES
    Led = auto() # led
    Sensor = auto() # sensor

    # LITERALES
    PinAnalogico = auto() # A0 ... A5
    PinDigital = auto() # D0 ... D13
    Numero = auto() # entero o flotante: 100, 3.3
    Identificador = auto() # nombre de variable del usuario

    # CONTROL INTERNO
    Error = auto() # carácter no reconocido

@dataclass
class Token:
    tipo: TokenTypes
    valor: Any
    linea: int
    columna: int
    posicion: int
```


4.2 Lógica del analizador léxico

El corazón del analizador se encuentra en el archivo `lexer.py`, donde se implementa la clase principal que recorre la cadena fuente carácter por carácter. El método de escaneo evalúa la entrada actual y determina si corresponde a un delimitador simple, a un operador relacional compuesto, o si debe ignorar la secuencia tratándose de espacios en blanco o comentarios.

Fragmento de `src/lexer.py` (Bucle principal):

```
def escanear_token(self) -> None:
    c = self.avanzar()

    # Delimitadores
    if c == ';': self.añadir_token(TokenTypes.PuntoYComa)
    elif c == '(': self.añadir_token(TokenTypes.ParentesisI)

    # Operadores relacionales
    elif c == '>':
        self.añadir_token(TokenTypes.MayorIgualQ if self.coincidir('=') else
TokenTypes.MayorQ)

    # Comentarios y descartes
    elif c == '#':
        self._procesar_comentario_numeral()
    elif c in (' ', '\r', '\t'):
        pass
```

4.3 Reglas léxicas para literales e identificadores

Para los elementos dinámicos más complejos, como los identificadores, las cadenas de texto y los números, el analizador delega la tarea a métodos auxiliares especializados. Por ejemplo, el procesador de identificadores extrae secuencias alfanuméricas completas y verifica primero si corresponden a pines de la placa de Arduino, y posteriormente si coinciden con alguna palabra clave definida en el vocabulario.

Fragmento de src/lexer.py (Procesamiento de identificadores):

```
def _procesar_identificador(self) -> None:
    while self.mirar_siguiete().isalnum() or self.mirar_siguiete() == '_':
        self.avanzar()

    texto = self.fuente[self.inicio:self.actual]

    # Pin analógico: A seguida de un dígito 0-5
    if len(texto) == 2 and texto[0] == 'A' and texto[1] in '012345':
        self.añadir_token(TokenTypes.PinAnalogico, texto)
        return

    # Búsqueda en el vocabulario (insensible a mayúsculas)
    tipo = self.PALABRAS_CLAVE.get(texto.lower(), TokenTypes.Identificador)
    self.añadir_token(tipo, texto)
```

4.4 Manejo y reporte de errores

El manejo de errores léxicos se realiza de forma tolerante para evitar la interrupción prematura del análisis. Cuando el analizador encuentra un símbolo desconocido o una estructura malformada, registra un mensaje detallado con la línea y columna exactas en una lista interna, emite un token de tipo error, y avanza al siguiente carácter. Al finalizar, el módulo errors.py es el encargado de presentar un reporte limpio en la consola de salida.

Fragmento de src/lexer.py (Captura de error) y src/errors.py:

```
    else:
        msg = f"Caracter desconocido '{c}' en linea {self.linea}, col {self.columna - 1}."
        self.errores.append(msg)
        self.añadir_token(TokenTypes.Error, c)

# -----
# En errors.py
def reportar_errores(errores: List[str]) -> None:
    if errores:
        imprimir_encabezado("ERRORES LÉXICOS DETECTADOS")
        for error in errores:
            print(f" [!] {error}")
```

Sección 5 — Demostración Funcional

Las siguientes capturas muestran la ejecución del lexer desde la terminal. Cada escenario fue ejecutado utilizando el comando `python src/main.py`.

5.1 Ejecución con programa válido

Se tokeniza el archivo `tests/valid/programa1.ard`, el cual implementa el control de un LED con parpadeo y la lectura de un sensor. El código cumple con las reglas gramaticales y no contiene errores léxicos.

Archivo de entrada — `tests/valid/programa1.ard`:

```
# ArduinoEspañol — Programa válido 1
# Control de LED con parpadeo y lectura de sensor
```

```
configurar pin 13 como salida
configurar pin A0 como entrada
configurar pin D9 como salida
```

```
// Encender LED integrado de la placa
encender led 13
esperar 500 ms
```

```
apagar led 13
esperar 500 ms
```

```
// Leer sensor de luz y reaccionar
leer sensor en A0
```

```
si A0 > 512 entonces
    encender led 13
sino
    apagar led 13
fin_si
```

```
// Ciclo de parpadeo 3 veces
repetir 3 veces
    encender led 13
    esperar 200 ms
    apagar led 13
    esperar 200 ms
fin_repetir
```

```
/* Apagado seguro al terminar */
apagar led 13
```

Ejecución y salida: \$ python src/main.py tests/valid/programa1.ard

```
PS C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol> py src/main.py tests/valid/programa1.ard
Archivo : C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol\tests\valid\programa1.ard

=====
ARDUINOESPAÑOL - TOKENS GENERADOS
=====
[ 1] Token(NuevaLinea      '\n'           linea=1, col=37)
[ 2] Token(Configurar      'configurar'   linea=2, col=1)
[ 3] Token(Pin             'pin'          linea=2, col=12)
[ 4] Token(Numero          13                    linea=2, col=16)
[ 5] Token(Como            'como'         linea=2, col=19)
[ 6] Token(Salida          'salida'       linea=2, col=24)
[ 7] Token(NuevaLinea      '\n'           linea=2, col=30)
[ 8] Token(Configurar      'configurar'   linea=3, col=1)
[ 9] Token(Pin             'pin'          linea=3, col=12)
[10] Token(PinAnalogico     'A0'           linea=3, col=16)
[11] Token(Como            'como'         linea=3, col=19)
[12] Token(Entrada          'entrada'      linea=3, col=24)
[13] Token(NuevaLinea      '\n'           linea=3, col=31)
[14] Token(NuevaLinea      '\n'           linea=4, col=1)
[15] Token(Encender        'encender'     linea=5, col=1)
[16] Token(Led             'led'          linea=5, col=10)
[17] Token(Numero          13                    linea=5, col=14)
[18] Token(NuevaLinea      '\n'           linea=5, col=16)
[19] Token(Esperar          'esperar'      linea=6, col=1)
[20] Token(Numero          500                    linea=6, col=9)
[21] Token(Ms              'ms'           linea=6, col=13)
[22] Token(NuevaLinea      '\n'           linea=6, col=15)
[23] Token(Apagar           'apagar'       linea=7, col=1)
[24] Token(Led             'led'          linea=7, col=8)
[25] Token(Numero          13                    linea=7, col=12)
[26] Token(NuevaLinea      '\n'           linea=7, col=14)
[27] Token(NuevaLinea      '\n'           linea=8, col=1)
[28] Token(Leer             'leer'         linea=9, col=1)
[29] Token(Sensor           'sensor'       linea=9, col=6)
[30] Token(En               'en'           linea=9, col=13)
[31] Token(PinAnalogico     'A0'           linea=9, col=16)
[32] Token(NuevaLinea      '\n'           linea=9, col=18)
[33] Token(Si               'si'           linea=10, col=1)
[34] Token(PinAnalogico     'A0'           linea=10, col=4)
[35] Token(MayorQ          '>'            linea=10, col=7)
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol> py src/main.py tests/valid/programa1.ard
[ 41] Token(Numero          13          linea=11, col=18)
[ 42] Token(NuevaLinea      '\n'        linea=11, col=20)
[ 43] Token(SiNo             'sino'    linea=12, col=1)
[ 44] Token(NuevaLinea      '\n'        linea=12, col=5)
[ 45] Token(Apagar           'apagar'  linea=13, col=5)
[ 46] Token(Led              'led'    linea=13, col=12)
[ 47] Token(Numero          13          linea=13, col=16)
[ 48] Token(NuevaLinea      '\n'        linea=13, col=18)
[ 49] Token(FinSi            'fin_si'  linea=14, col=1)
[ 50] Token(NuevaLinea      '\n'        linea=14, col=7)
[ 51] Token(FinalCodigo      ''         linea=15, col=1)

[OK] Sin errores léxicos.

=====
RESUMEN POR CATEGORÍA
=====
Numero          7 ocurrencia(s)
Led              4 ocurrencia(s)
PinAnalogico     3 ocurrencia(s)
Configurar       2 ocurrencia(s)
Pin              2 ocurrencia(s)
Como             2 ocurrencia(s)
Encender         2 ocurrencia(s)
Apagar           2 ocurrencia(s)
Salida           1 ocurrencia(s)
Entrada          1 ocurrencia(s)
Esperar          1 ocurrencia(s)
Ms               1 ocurrencia(s)
Leer             1 ocurrencia(s)
Sensor           1 ocurrencia(s)
En               1 ocurrencia(s)
Si               1 ocurrencia(s)
MayorQ           1 ocurrencia(s)
Entonces         1 ocurrencia(s)
SiNo             1 ocurrencia(s)
FinSi            1 ocurrencia(s)
TOTAL           36 token(s)
```

Captura 5.1 — Tokenización completa del programa programa1.ard sin errores léxicos.

5.2 Ejecución con errores léxicos

Se tokeniza el archivo tests/invalid/errores1.ard, el cual fue diseñado intencionalmente para contener símbolos no permitidos por la gramática de ArduinoEspañol, como el carácter @, cadenas de texto sin comilla de cierre y operadores incompletos.

Archivo de entrada — tests/invalid/errores1.ard:

```
# ArduinoEspañol — Errores léxicos 1
```

```
configurar pin 13 como salida
```

```
encender led @13
```

```
si A0 = 512 entonces
```

```
    encender led 13
```

```
fin_si
```

```
apagar led 13 &
```

Ejecución y salida: \$ python src/main.py tests/invalid/errores1.ard

```
PS C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol> py src/main.py tests/invalid/errores1.ard
Archivo : C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol\tests\invalid\errores1.ard
```

=====

ARDUINOESPAÑOL – TOKENS GENERADOS

=====

[1]	Token(NuevaLinea	'\n'	linea=1, col=37)
[2]	Token(Configurar	'configurar'	linea=2, col=1)
[3]	Token(Pin	'pin'	linea=2, col=12)
[4]	Token(Numero	13	linea=2, col=16)
[5]	Token(Como	'como'	linea=2, col=19)
[6]	Token(Salida	'salida'	linea=2, col=24)
[7]	Token(NuevaLinea	'\n'	linea=2, col=30)
[8]	Token(NuevaLinea	'\n'	linea=3, col=1)
[9]	Token(Encender	'encender'	linea=4, col=1)
[10]	Token(Led	'led'	linea=4, col=10)
[11]	Token(Error	'@'	linea=4, col=14)
[12]	Token(Numero	13	linea=4, col=15)
[13]	Token(NuevaLinea	'\n'	linea=4, col=17)
[14]	Token(Si	'si'	linea=5, col=1)
[15]	Token(PinAnalogico	'A0'	linea=5, col=4)
[16]	Token(Error	'='	linea=5, col=7)
[17]	Token(Numero	512	linea=5, col=9)
[18]	Token(Entonces	'entonces'	linea=5, col=13)
[19]	Token(NuevaLinea	'\n'	linea=5, col=21)
[20]	Token(Encender	'encender'	linea=6, col=5)
[21]	Token(Led	'led'	linea=6, col=14)
[22]	Token(Numero	13	linea=6, col=18)
[23]	Token(NuevaLinea	'\n'	linea=6, col=20)
[24]	Token(FinSi	'fin_si'	linea=7, col=1)
[25]	Token(NuevaLinea	'\n'	linea=7, col=7)
[26]	Token(Apagar	'apagar'	linea=8, col=1)
[27]	Token(Led	'led'	linea=8, col=8)
[28]	Token(Numero	13	linea=8, col=12)
[29]	Token(Error	'&'	linea=8, col=15)
[30]	Token(NuevaLinea	'\n'	linea=8, col=16)
[31]	Token(FinalCodigo	''	linea=9, col=1)


```
=====
ERRORES LÉXICOS DETECTADOS
=====
[!] Caracter desconocido '@' en línea 4, col 14.
[!] Símbolo '=' suelto en línea 5, col 7: use '==' para comparar.
[!] Caracter desconocido '&' en línea 8, col 15.

Total de errores: 3

=====
RESUMEN POR CATEGORÍA
=====
Numero          5 ocurrencia(s)
Led              3 ocurrencia(s)
Error            3 ocurrencia(s)
Encender         2 ocurrencia(s)
Configurar       1 ocurrencia(s)
Pin              1 ocurrencia(s)
Como             1 ocurrencia(s)
Salida           1 ocurrencia(s)
Si               1 ocurrencia(s)
PinAnalogico     1 ocurrencia(s)
Entonces         1 ocurrencia(s)
FinSi            1 ocurrencia(s)
Apagar           1 ocurrencia(s)
-----
TOTAL            22 token(s)

PS C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol> 
```

Captura 5.2 — El lexer detecta los símbolos inválidos y la cadena malformada, reporta su ubicación exacta (línea y columna) y continúa procesando el resto del programa sin detenerse abruptamente

5.3 Caso con caracteres no reconocidos y bloques sin cerrar

Se tokeniza el archivo `tests/invalid/errores2.ard`, el cual expone fallos como caracteres inválidos (`$`, `%`) en medio de valores numéricos y un bloque de comentario de múltiples líneas que nunca se cierra.

Archivo de entrada — `tests/invalid/errores2.ard`:

```
# ArduinoEspañol — Errores léxicos 2
configurar pin D5 como salida

mover servo D5 a 90$ grados
esperar 100 % ms
/* comentario sin cierre
```

```
PS C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol> py src/main.py tests/invalid/errores2.ard
Archivo : C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol\tests\invalid\errores2.ard
```

```
=====
ARDUINOESPAÑOL -- TOKENS GENERADOS
=====
```

```
[ 1] Token(NuevaLinea      '\n'           linea=1, col=37)
[ 2] Token(Configurar      'configurar'  linea=2, col=1)
[ 3] Token(Pin              'pin'        linea=2, col=12)
[ 4] Token(PinDigital       'D5'         linea=2, col=16)
[ 5] Token(Como             'como'       linea=2, col=19)
[ 6] Token(Salida           'salida'     linea=2, col=24)
[ 7] Token(NuevaLinea      '\n'           linea=2, col=30)
[ 8] Token(NuevaLinea      '\n'           linea=3, col=1)
[ 9] Token(Mover            'mover'      linea=4, col=1)
[10] Token(Servo            'servo'       linea=4, col=7)
[11] Token(PinDigital       'D5'         linea=4, col=13)
[12] Token(A                'a'          linea=4, col=16)
[13] Token(Numero           90                   linea=4, col=18)
[14] Token(Error            '$'                  linea=4, col=20)
[15] Token(Grados          'grados'    linea=4, col=22)
[16] Token(NuevaLinea      '\n'           linea=4, col=28)
[17] Token(Esperar          'esperar'   linea=5, col=1)
[18] Token(Numero           100                  linea=5, col=9)
[19] Token(Error            '%'                  linea=5, col=13)
[20] Token(Ms               'ms'         linea=5, col=15)
[21] Token(NuevaLinea      '\n'           linea=5, col=17)
[22] Token(FinalCodigo      ''              linea=7, col=2)
```

```
=====
ERRORES LÉXICOS DETECTADOS
=====
[!] Caracter desconocido '$' en línea 4, col 20.
[!] Caracter desconocido '%' en línea 5, col 13.
[!] Comentario de bloque sin cerrar, inicio en línea 7.

Total de errores: 3

=====
RESUMEN POR CATEGORÍA
=====
PinDigital          2 ocurrencia(s)
Numero              2 ocurrencia(s)
Error               2 ocurrencia(s)
Configurar          1 ocurrencia(s)
Pin                 1 ocurrencia(s)
Como                1 ocurrencia(s)
Salida              1 ocurrencia(s)
Mover               1 ocurrencia(s)
Servo               1 ocurrencia(s)
A                  1 ocurrencia(s)
Grados              1 ocurrencia(s)
Esperar             1 ocurrencia(s)
Ms                  1 ocurrencia(s)
-----
TOTAL                16 token(s)

PS C:\Users\cowel\OneDrive\Escritorio\ArduinoEspañol> 
```

Ejecución y salida: \$ python src/main.py tests/invalid/errores2.ard

Sección 6 — Conclusiones

Gómez Rubio Iram Said

"A lo largo del desarrollo de este proyecto, mi principal aprendizaje se centró en el diseño y la estructuración del vocabulario del lenguaje en el módulo `tokens.py`. Comprender cómo una enumeración estricta (`TokenTypes`) y una clase contenedora (`Token`) actúan como el puente de comunicación entre el analizador léxico y las futuras fases del compilador fue fundamental. Asimismo, afrontar el reto de mapear palabras clave e instrucciones operacionales en español de forma insensible a mayúsculas y minúsculas mediante diccionarios hash optimizados me permitió dimensionar la importancia de la eficiencia en el software de base."

Reyes Avalos Joshua Emmanuel

"Mi participación en el núcleo del Lexer me dejó una comprensión profunda de cómo se procesa un archivo fuente a su nivel más primitivo. Implementar manualmente el bucle de escaneo carácter por carácter y mantener un control estricto sobre los punteros de navegación (posición, línea, columna) me enseñó a manejar la sincronización física del archivo. Un gran reto técnico fue diseñar autómatas lógicos secuenciales que fueran capaces de ignorar los espacios en blanco y descartar múltiples formatos de comentarios (`#`, `//` y `/* ... */`) de manera exacta sin corromper el rastreo de las coordenadas del código."

Contreras Morales Ricardo Axel

"Desarrollar la lógica para el reconocimiento de literales específicos del hardware de Arduino fue una experiencia sumamente enriquecedora. Tuve que diseñar algoritmos de extracción para capturar de manera exacta números enteros o flotantes, cadenas de texto delimitadas por comillas y, de forma muy especial, los rangos restringidos de los pines analógicos (A0-A5) y digitales (D0-D13). Esto me ayudó a dominar el concepto de análisis prospectivo (`lookahead`), evaluando el carácter siguiente antes de consumir el flujo, garantizando que un identificador común no fuera confundido con una regla léxica estricta."

Serrano Vargas Aldo Antonio

"Mi enfoque estuvo orientado al diseño del sistema de gestión y reporte de incidencias en los módulos `errors.py` y `main.py`. El mayor aprendizaje fue asimilar que un buen compilador no debe aplicar una política de pánico y detenerse ante el primer fallo; en su lugar, debe ser resiliente. Lograr que nuestro lexer capture caracteres inválidos o símbolos desconocidos (como `$` o `@`), registre su ubicación exacta con línea y columna, y continúe analizando el resto del archivo fue un logro crítico que aporta un enorme valor didáctico al usuario final del lenguaje."

Trabajo futuro (esta sección totalmente opcional)

- Extender el lexer para reconocer cadenas multilínea con `"""`.
- Agregar soporte para números en notación científica (`1.5e10`).
- Integrar el analizador léxico con un analizador sintáctico (`parser`) para validar también la gramática del lenguaje.
- Generar un reporte HTML coloreado de los tokens por categoría.